

Pipeline MLOps

Despliegue automatizado de un clasificador Iris en AWS Lambda

Metodologías de Desarrollo y Despliegue de Aplicaciones para Ciencia de Datos

Richard Makovs Seleznova | Grado en Ciencia de Datos e IA — VIU | 6 de mayo de 2026

Introducción

Proyecto elegido: Pipeline MLOps con despliegue automatizado de un modelo de clasificación.

Objetivo: diseñar e implementar un pipeline DevOps/MLOps completo aplicando metodologías ágiles, contenedores, CI/CD e infraestructura como código.

El problema y la elección

Dataset: Iris

150 muestras, 4 features, 3 clases.

Modelo: Random Forest

scikit-learn, n_estimators=100, random_state=42.

Stack tecnológico

Modelo y API

scikit-learn, FastAPI, Pydantic, Mangum

Testing

pytest, pytest-cov (cobertura del 85%)

Contenedor

Docker sobre imagen base de AWS Lambda Python 3.12

CI/CD

GitHub Actions con dos workflows separados (CI y CD)

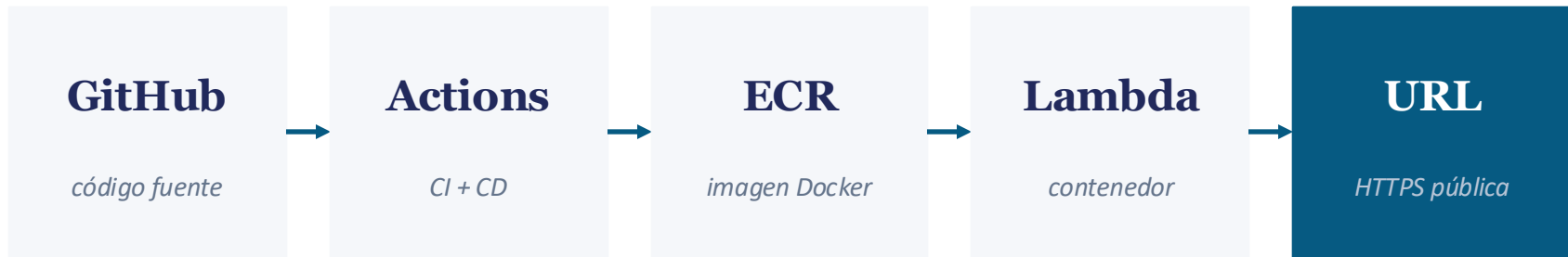
Cloud

AWS Lambda, ECR, IAM, CloudWatch (región eu-west-1)

IaC

Terraform 1.5 con provider AWS 5.x

Arquitectura del sistema



Cada push a la rama main dispara el pipeline. Validación de tests, construcción de la imagen, publicación en ECR y actualización de Lambda. Tiempo total de commit a producción: menos de 5 minutos.

Decisiones técnicas

FastAPI con Pydantic

Validación automática de inputs sin código manual. Documentación interactiva en /docs. Estándar de facto en MLOps moderno.

AWS Lambda en lugar de App Runner

App Runner cerró a nuevos clientes el 30/04/2026. Lambda con contenedor ofrece HTTPS público vía Function URL y entra en el free tier.

Terraform como IaC

Infraestructura reproducible y versionada. Permite destruir y recrear el entorno en minutos sin configuración manual.

CI y CD en workflows separados

CI corre en push y pull request (feedback rápido). CD solo en push a main, evita despliegues accidentales desde ramas en desarrollo.

Demo

Tests · Pipelines · API en producción

Metodología ágil

Tres sprints documentados con tablero Kanban en GitHub Projects, 13 user stories en formato Scrum, todas cerradas como Done.

Sprint 1

Desarrollo local

Modelo, API, validación Pydantic,
suite de tests con cobertura 85%

Sprint 2

Contenedorización + CI

Dockerfile optimizado, workflow CI,
ECR via Terraform, umbral de
cobertura

Sprint 3

Despliegue + docs

Lambda + Function URL, workflow
CD, migración App Runner→Lambda,
README, memoria

Conclusiones

Pipeline funcional. Cada commit aprobado en main llega a producción.

Decisiones documentadas y justificadas.

Aprendizaje principal.

Mejoras futuras: OIDC en lugar de claves estáticas, build Docker multi-etapa, restricción IAM al mínimo privilegio y observabilidad con métricas y trazas.